

CS1001: Introduction to Computing

Module 1 and 2: Notes by Sudeep K S, September 2026

Set 2: C Operators

- **Arithmetic Operators:** +, -, *, /, % and their variants (++ , --, +=, -=, *= etc)

The first 4 are basic arithmetic operations:

They are used to Add, Subtract, Multiply and Divide two numbers.

Example: `c = a * b;` (assigns the value a multiplied by b to the variable c).

When applied to integers, division operator gives value of the quotient only.

For example, when `a = 7` and `b = 4`, if both a and b are of integer type, `a / b` gives 1.

% is used for 'modulo'. `a % b` is read as 'a modulo b', it returns the remainder when a is divided by b. Example: `c = 13 % 5;` assigns value 3 to c.

`++` and `--` are unary operators – means they operate on a single variable.

`++` increments the value of a variable by 1; `--` decrements the variable value by 1.

Example: `a = 5; a++;` (now a is 6)

`b = 3; b--;` (now b becomes 2).

Operators `+=`, `-=`, `*=` and `/=` are used to 'add and assign', 'subtract and assign', 'multiply and assign' and 'divide and assign' respectively.

`a += b;` This is same as `a = a + b;`

`a -= b;` is same as `a = a - b;`

`a *= b;` is same as `a = a * b;`

`a /= b;` is same as `a = a / b;`

Example: `a = 15; b = 10; a += 5; b /= 5;`

Now a becomes 20 and b becomes 2.

Exercise: `a = 5; b = 8; a++; a *= 6; b--;` `a -= a % b;` What is the value of 'a' now?

- **Relational Operators:** <, <=, >, >=, ==, !=

These are 'less than', 'less than or equal to', 'greater than', 'greater than or equal to', 'equal to' and 'not equal to' respectively.

Example: `a = b < c;` (assigns 1 or true to a if b is less than c; 0 otherwise).

In the above example, if 'a' is defined as 'int', it takes value 1 if the condition is true, and becomes 0 otherwise.

If 'a' is defined as 'bool', it takes 'true' or 'false' value but it is also equivalent to 1 (true) / 0 (false).

If we print the value of 'a', we get 1 or 0 only, even if 'a' is defined as 'bool'.

Another example:

`a = (b != c);` ('a' becomes True / 1 if b is not equal to c; 0 otherwise).

Exercise: Try running the below program. (What do you think will be the output?)

```
#include <stdio.h>

int main() {
    int a = 7, b = 4;

    // testing operators
    printf("a < b: %d\n", a < b);
    printf("a > b: %d\n", a > b);
    printf("a <= b: %d\n", a <= b);
    printf("a >= b: %d\n", a >= b);
    printf("a == b: %d\n", a == b);
    printf("a != b: %d\n", a != b);

    return 0;
}
```

- **Logical Operators:** OR (`|`), AND (`&`) and NOT (`!`)

Example: `a = b < c || d == 0;`

Value of `a` becomes 1 (true) if either of the two parts is true: `b < c` OR `d == 0`.

Recollect that `A OR B` is true if at least one of them is true (else it is false).

Similarly, `A AND B` is true if both `A` and `B` are true, it is false otherwise.

`a = b < c && d == 0; // true (1) if b < c AND d == 0; false otherwise`

NOT (`!`) just negates the value. It is applied on a single operand / condition.

`a = !(b < c); // a becomes true if b is not less than c; false otherwise`

Exercises:

```
int a = 6; int b = 10; int c = (a / b) && (a < b);
printf("c is: %d\n", c);
```

What will be the value of `c` that gets printed? Hint: `a / b` is 0; `a < b` is 1 (true)

Now, what if we replace that '`c`' line with the following:

```
c = (a / b) || (a < b && !(b < 20));
```

and then print `c`?

- **Bitwise AND and OR operators:** '`&`' and '`|`'

These operators work like arithmetic operators, though they do AND/OR.

'`&`' does AND between every corresponding bit; and '`|`' does OR similarly.

For example, when we have: `a = 5; b = 9; c = a & b; d = a | b;`

`a` is '0101' in binary and `b` is '1001'; so doing AND, we have

```
0 1 0 1 &
1 0 0 1
-----
```

0 0 0 1 (recollect how AND works. 0 AND 1 is 0; 1 AND 0 is 0; 1 AND 1 is 1)

So `c` gets value 1 (because 0001 corresponds to 1).

Exercise: What will be `d`? Also, what is `a & b` and `a | b` if `a = 10` and `b = 17`?